



Research and implementation blueprint for your SJSU author-persona / RAG project

Executive assessment

Based on what you described, I would **not** treat the project as one monolithic “train a model to sound like an author” problem. The cleanest research design is to separate three things that are currently at risk of getting mixed together:

1. **What the system knows** — facts, topics, entities, relationships, prior statements, and other information that should ground an answer.
2. **How a particular author writes** — punctuation, syntax, vocabulary, cadence, emoji use, sentence length, discourse patterns, and other stylistic signals.
3. **Which source data are genuinely representative of that author** — as opposed to advertisements, affiliate boilerplate, reposts, agency-written promotions, duplicated campaigns, platform artifacts, or other contaminated material.

That separation is important because authorship research repeatedly finds that **topic/content and writing style can become confounded**, while recent LLM-personalization research shows that fluent models still do not reliably reproduce the implicit writing style of ordinary authors from examples alone. ¹

The architecture I would presently recommend as the center of the project is therefore:

Ontology-grounded content/persona RAG + separately retrieved author-style exemplars + an optional PEFT/LoRA style adapter, evaluated against substantially simpler baselines.

That gives your professor a genuine ontology-oriented RAG architecture while preserving a scientifically defensible comparison against ordinary RAG and prompting. OG-RAG, published at EMNLP 2025, explicitly grounds retrieval in a domain ontology and represents knowledge as ontology-grounded hyperedges; the authors report substantial gains over their baselines in fact recall, response correctness, attribution speed, and fact-based reasoning. ² At the same time, a separate EMNLP 2025 study found that relatively simple retrieve-then-read approaches can match or outperform more complicated RAG pipelines under controlled conditions, which means your project needs to **demonstrate that its extra architecture complexity earns its keep**, rather than assuming more sophisticated RAG is automatically better. ³

There is one important evidence boundary: **I have not yet inspected the actual PDFs in C:\Users\okevi\SJSU\DATA298\298P\docs\research, your proposal/workbook, or the current repository from this chat.** A Windows path alone does not expose those files to this conversation. I surfaced the available Remote Desktop Commander and GitHub integrations because they would allow the local research directory and repository to become directly inspectable once connected. Consequently, this

report is a research-backed architecture and project-context design based on everything in your message; it is **not pretending to have validated unseen proposal diagrams or papers**.

The goal should now be to turn your repository into the **canonical project memory** for you, me, Cursor, and any other coding agent. ChatGPT Projects can keep files, instructions, and project chats together and use that material as context; moving a chat into a Project causes it to inherit that Project's instructions and file context. ⁴ Cursor similarly supports repository-scoped instructions through `.cursor/rules`, `AGENTS.md`, commands, and project-scoped memories. ⁵ The durable truth, however, should live in **version-controlled project files rather than only in either product's conversational memory**.

My high-level recommendation is therefore:

Area	Recommended direction
Project memory	Version-controlled <code>docs/project/</code> + <code>AGENTS.md</code> + Cursor rules
Research papers	Build a manifest and evidence/claims matrix; do not leave them as an unstructured PDF folder
Raw social data	Immutable raw layer; never clean destructively
Ad/promotion removal	Label and score first, then construct multiple derived corpora instead of permanently deleting
Style modeling	Separate style conditioning from factual grounding
Baseline	Few-shot/exemplar prompting and ordinary vector RAG
Main architecture	Ontology-grounded hybrid RAG + author exemplar retrieval
Advanced architecture	Same ontology RAG + PEFT/LoRA style model
Evaluation	Separate retrieval, factuality, style fidelity, copying/privacy, cost, and latency
Research design	Matched-data/model/token-budget ablations
Agent workflow	Every major planning/coding session updates <code>PROJECT_STATE</code> , <code>PLAN</code> , decisions, and experiment records

This also aligns well with SJSU's emphasis on completing the total analytics lifecycle and producing quality project artifacts in its current Applied Data Science project assessment materials. ⁶

A canonical context system for ChatGPT and Cursor

The biggest improvement you can make **before implementing more architecture** is to establish one explicit source of truth. Otherwise, the proposal says one thing, the workbook says another, Cursor infers something from code, our chats contain later decisions, and eventually nobody—including you—knows which version is authoritative.

Cursor's current documentation supports project rules under `.cursor/rules`; its CLI also reads a root `AGENTS.md`, while reusable Markdown commands can live under `.cursor/commands`. Cursor also has project-scoped memories, but those should complement rather than replace reproducible repository documentation. ⁵ ChatGPT Projects likewise support project instructions, files, saved responses, and project chats as context, making them useful for keeping our research and planning together. ⁷

Recommended repository context structure

I would converge your project toward something similar to this:

```
298P/
|
├── AGENTS.md
├── README.md
├── .gitignore
|
├── .cursor/
|   ├── rules/
|   |   ├── 00-project-context.mdc
|   |   ├── 10-architecture.mdc
|   |   ├── 20-data-governance.mdc
|   |   ├── 30-research-evidence.mdc
|   |   ├── 40-testing-evaluation.mdc
|   |   └── 50-agent-workflow.mdc
|   └── commands/
|       ├── plan-task.md
|       ├── review-architecture.md
|       ├── run-experiment.md
|       └── update-project-state.md
|
├── docs/
|   ├── project/
|   |   ├── PROJECT_CONTEXT.md
|   |   ├── PROJECT_STATE.md
|   |   ├── PLAN.md
|   |   ├── DECISION_LOG.md
|   |   ├── ASSUMPTIONS.md
|   |   └── GLOSSARY.md
|   |
|   ├── architecture/
|   |   ├── ARCHITECTURE.md
|   |   ├── ARCHITECTURE_BASELINE.md
|   |   ├── ARCHITECTURE_ONTOLOGY_RAG.md
|   |   ├── ARCHITECTURE_PEFT.md
|   |   └── EXPERIMENT_MATRIX.md
|   |
|   └── data/
```

```

├── DATA_DICTIONARY.md
├── DATA_PROVENANCE.md
├── CLEANING_SPEC.md
├── PROMOTION_LABELING.md
├── DATA_SPLITS.md
├── evaluation/
│   ├── EVALUATION_PLAN.md
│   ├── STYLE_METRICS.md
│   └── RAG_METRICS.md
├── research/
│   ├── INDEX.md
│   ├── papers/ # or your current PDFs
│   ├── PAPER_MANIFEST.csv
│   ├── CLAIMS_MATRIX.csv
│   ├── RESEARCH_SYNTHESIS.md
│   └── GAPS.md
├── data/
│   ├── raw/ # never modified
│   ├── interim/
│   ├── processed/
│   └── annotations/
├── ontology/
│   ├── ontology.ttl
│   ├── schema.md
│   ├── mappings/
│   └── versions/
├── src/
├── tests/
├── configs/
├── experiments/
│   ├── registry.csv
│   └── results/
└── notebooks/

```

You do **not** need all of that in one commit. The structural principle matters more than the exact names.

The files that should become authoritative

`PROJECT_CONTEXT.md` should answer the stable questions: what the project does, its research question, target users, inputs, outputs, constraints, success criteria, terminology, and scope exclusions.

PROJECT_STATE.md should answer the changing questions:

```
# Project State

Last updated:
Current branch:
Last verified commit:

## Current objective

## Current project phase

## Confirmed decisions

## Architecture currently implemented

## Dataset currently in use

## Experiments completed

## Current results

## Known problems

## Open research questions

## Active implementation task

## Next planned task

## Risks / blockers
```

PLAN.md should contain the **agreed path**, rather than a generic to-do list. When we use a reasoning/ planning session to decide, for example, that promotion detection happens before style profiling or that ontology retrieval will be an experimental branch, that decision belongs here.

DECISION_LOG.md should preserve *why* we changed direction:

```
ADR-007
Date:
Decision:
Problem:
Alternatives considered:
Evidence:
Affected files:
```

Expected consequence:

Status: proposed | accepted | superseded

This is how you solve the specific problem you raised about deep-thinking/planning work disappearing into chat history. The chat develops the plan; **the repository records the resulting decision.**

Source-of-truth hierarchy

I would explicitly put the following hierarchy in `AGENTS.md`:

When sources disagree:

1. Current `PROJECT_CONTEXT.md` defines project scope.
2. Accepted entries in `DECISION_LOG.md` define architectural decisions.
3. `PROJECT_STATE.md` defines current implementation status.
4. Architecture/data/evaluation specifications define subsystem behavior.
5. Source code and tests show what is actually implemented.
6. Research papers provide external evidence, not automatic project requirements.
7. Proposal/workbook are historical design inputs and may be superseded.
8. Chat history and agent memory are supporting context, not authoritative truth.

That single rule will prevent a tremendous amount of agent drift.

A root `AGENTS.md` I would give both coding agents

A useful starting point would be:

Agent Instructions

Project objective

Implement and experimentally evaluate a personalized language-generation system that separates factual/persona grounding from author-style modeling.

Required context before architectural work

Read:

- docs/project/PROJECT_CONTEXT.md
- docs/project/PROJECT_STATE.md
- docs/project/PLAN.md
- docs/project/DECISION_LOG.md
- docs/architecture/ARCHITECTURE.md
- docs/evaluation/EVALUATION_PLAN.md

For data work also read:

- docs/data/DATA_DICTIONARY.md
- docs/data/CLEANING_SPEC.md
- docs/data/DATA_PROVENANCE.md

Core invariants

- Never modify raw source data in place.
- Keep factual grounding separate from style conditioning.
- Preserve source provenance for retrieved knowledge.
- Do not silently change an accepted architecture decision.
- Do not use test data during training, prompt construction, retrieval-index construction, tuning, or threshold selection.
- Do not treat promotional-content filtering as ground truth unless it was manually validated.
- New architectural complexity must be compared with a simpler baseline.
- Research-driven implementation decisions must reference the relevant paper or decision record.

Planning protocol

Before a substantial task:

1. Read the current state and plan.
2. State assumptions.
3. Identify affected files.
4. Define acceptance tests.

After a substantial task:

1. Run relevant tests/evaluations.
2. Update PROJECT_STATE.md.
3. Update DECISION_LOG.md if a design decision changed.
4. Record experimental changes with model/data/config/git identifiers.
5. Update PLAN.md when the agreed execution path changes.

Cursor is specifically designed to reuse project rules for team-wide behavior, and its documentation recommends rules as persistent context rather than repeatedly re-explaining project expectations. ⁸

For ChatGPT, keeping this work in the same Project, saving major research conclusions as Project sources, and moving the architecture/repository discussion into that same Project will create a much stronger continuity layer than expecting unrelated chats to perfectly reconstruct one another. OpenAI's current Project documentation explicitly describes files, chats, and project instructions as reusable project context. ⁴

Turning the research-paper folder into usable project evidence

Your existing:

```
C:\Users\okevi\SJSU\DATA298\298P\docs\research
```

should not remain merely a directory of papers that humans or agents occasionally search.

It should become a **research evidence base**.

Inventory every paper before synthesizing it

Create `PAPER_MANIFEST.csv` with something close to:

Field	Purpose
<code>paper_id</code>	Stable internal ID
<code>filename</code>	Original file
<code>title</code>	Canonical title
<code>authors</code>	Authors
<code>year</code>	Publication year
<code>venue</code>	Journal/conference
<code>doi_or_arxiv</code>	Persistent identifier
<code>peer_reviewed</code>	yes/no/unknown
<code>topic</code>	RAG, ontology, personalization, etc.
<code>architecture_relevance</code>	Which architecture it affects
<code>data_relevance</code>	Which dataset/process it affects
<code>evaluation_relevance</code>	Which metric/experiment it supports
<code>notes</code>	Short synthesis
<code>sha256</code>	Reproducibility / duplicate detection

That gives us a way to ask questions like:

“Show me every peer-reviewed paper in my corpus supporting ontology-aware retrieval.”

rather than:

“Search these 46 PDFs and tell me what seems relevant.”

Build a claims matrix, not only summaries

`CLAIMS_MATRIX.csv` should contain evidence at a finer grain:

```

claim_id
paper_id
claim
evidence_location
method
dataset
metric
result
limitations
project_implication
confidence

```

For example:

```

CLAIM-042
PAPER-017
"Ontology-grounded retrieval improved fact recall relative to evaluated
baselines"
...
project_implication:
"Justifies evaluating ontology-grounded retrieval, but does not prove it
will improve style imitation."

```

That last distinction is crucial.

The ontology literature supports ontology/graph structure primarily for **retrieval, relationship reasoning, coherence, and grounding**. OG-RAG grounds document facts using an ontology and retrieves ontology-grounded hyperedges, while KG²RAG uses knowledge-graph relationships between chunks to improve retrieval organization and coherence. ⁹ These findings do **not** establish that ontology retrieval by itself makes a model sound more like a particular human author.

Likewise, recent style-personalization work finds that author imitation remains challenging and recommends multidimensional evaluation rather than assuming that fluent-looking output represents faithful style imitation. ¹⁰

Papers I would make sure are represented in your literature review

If they are not already in your research folder, these are especially important.

Research	Why it matters
OG-RAG: Ontology-grounded Retrieval-Augmented Generation	Closest direct justification for the professor's ontology-RAG suggestion; uses an explicit domain ontology and hypergraph representation. ¹¹

Research	Why it matters
Knowledge Graph-Guided RAG / KG²RAG	Shows how KG relationships can organize retrieved chunks and improve contextual coherence. ¹²
Microsoft GraphRAG	Useful comparison architecture for graph-derived retrieval and local/global querying. ¹³
Stronger Baselines for RAG with Long-Context LMs	Important methodological warning that complex retrieval pipelines must beat strong simple baselines under matched conditions. ³
Catch Me If You Can? Not Yet	Directly studies personalized author-style imitation and shows current LLM limitations; uses multiple style-evaluation approaches. ¹⁴
Interpreting Style Representations via Style-Eliciting Prompts	Recent ACL 2026 evidence that explicit representations/ descriptions of style can improve style steering relative to simply giving target texts. ¹⁵
Customizing LLM Generation Style using PEFT	Relevant justification for testing LoRA/PEFT as an author-style branch. ¹⁶
Analyzing Linguistic Differences between Owner and Staff Attributed Tweets	Direct warning that posts from one social-media account need not have one human author. ¹⁷

The final literature synthesis should therefore be organized around **project decisions**, not simply “Paper A said..., Paper B said...”.

I would use headings such as:

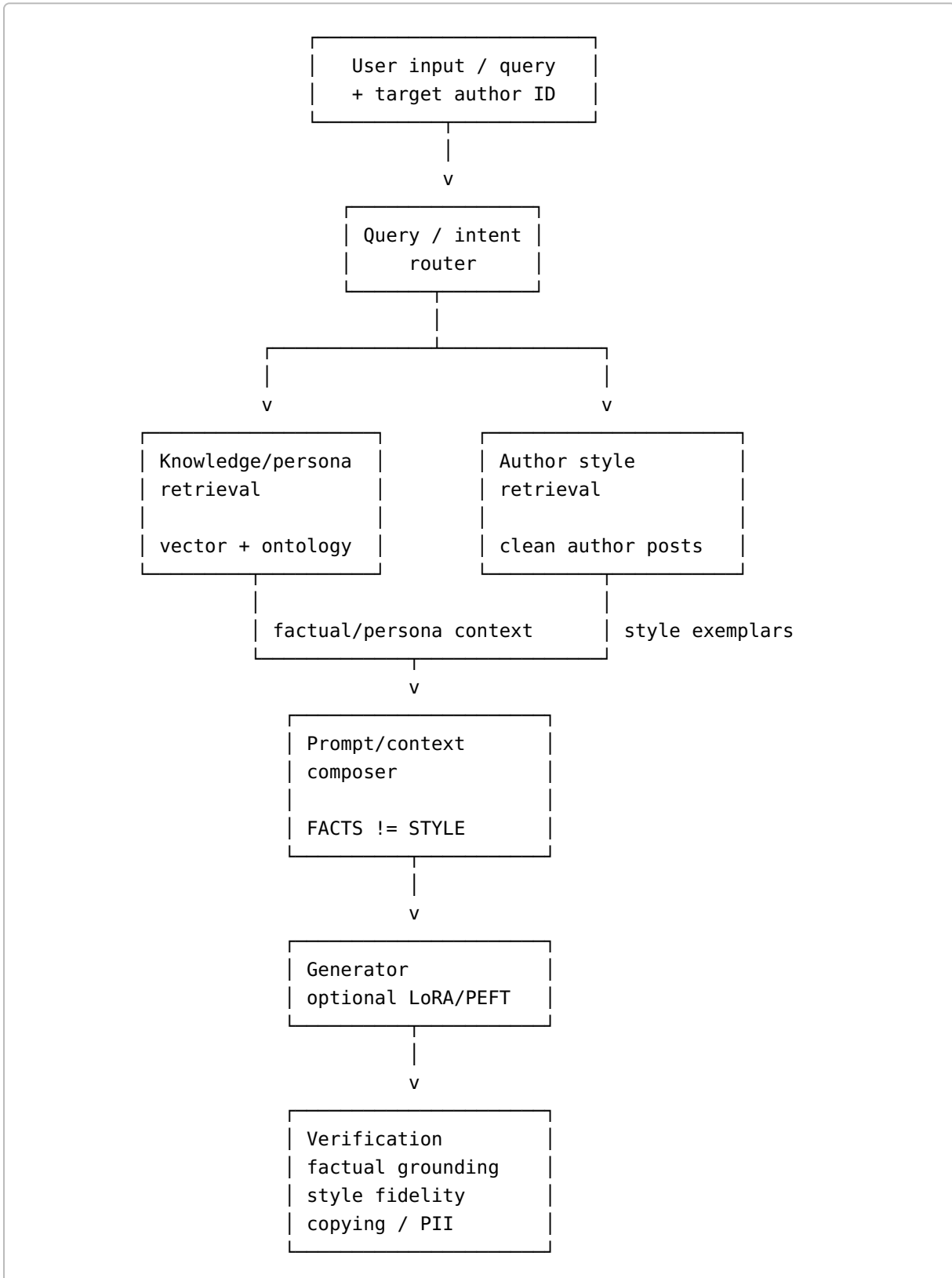
What should be retrieved?
What represents author style?
What contaminates author style?
When does fine-tuning outperform prompting?
When do knowledge graphs help?
What specifically makes RAG ontological?
How should style fidelity be evaluated?
How should factual grounding be evaluated?
What ethical constraints apply to social-media data?

That format will directly feed your architecture chapter and experimental design.

Architecture recommendation and proposal review framework

I would reframe the project around a **content/style separation principle**.

The core system



The important part is not the boxes. It is that **the factual retrieval context and stylistic demonstration context remain identifiable as two different signals.**

Architecture family to put into the project

I recommend four architectures rather than one vaguely evolving system:

Architecture	Purpose	Complexity	Role in project
A0 — Prompt/exemplar baseline	Few-shot clean author examples, no RAG	Low	Mandatory baseline
A1 — Style-RAG	Retrieve relevant author examples using vector similarity	Low–medium	Strong conventional baseline
A2 — Ontology-grounded hybrid RAG + Style-RAG	Structured persona/content retrieval plus separate style examples	Medium–high	Recommended primary architecture
A3 — A2 + PEFT/LoRA style adaptation	Ontology grounding plus trained style adaptation	High	Advanced experimental architecture

This design gives you an academically meaningful progression rather than three unrelated systems.

Why A0 is mandatory

Recent personalization work demonstrates that LLMs can produce fluent imitation while still missing subtle author-specific style, and it evaluates imitation using authorship attribution, verification, style matching, and AI-detection measures rather than a single surface-overlap metric. ¹⁴

Therefore, A0 answers:

How well can the same base model already do this with properly selected examples?

Without A0, an improvement from a complex architecture has no meaningful reference point.

Why A1 is useful

A1 tests whether **retrieving the right examples** is sufficient.

Instead of stuffing random author posts into the prompt, retrieve posts that are semantically or pragmatically relevant to the input. That can capture context-specific behavior such as how an author writes about sports versus politics, replies versus standalone posts, or positive versus critical statements.

That separation also helps address the longstanding authorship problem that topical content can be mistaken for stylistic identity. ¹⁸

Why A2 should probably become your centerpiece

A2 lets you make a substantive thesis claim:

Does explicit ontological organization of an author's/persona's knowledge improve retrieval, factual/persona consistency, and multi-hop reasoning compared with ordinary semantic retrieval, while a separate exemplar retriever controls writing style?

That is a much stronger research question than:

"Can I use GraphRAG?"

OG-RAG provides direct evidence that domain ontologies can structure retrieval more precisely, while KG²RAG demonstrates the usefulness of KG relationships between retrieved chunks. ⁹

Why A3 should be conditional rather than automatically "best"

Parameter-efficient fine-tuning has been studied as a mechanism for style customization, so LoRA/PEFT is a reasonable experimental branch. ¹⁶ But recent work still reports substantial difficulty in faithfully imitating everyday authors, meaning you should test rather than assume that fine-tuning solves personalization. ¹⁴

A3 also creates additional research risks:

- memorization versus generalizable style;
- overfitting to topics frequently discussed by the author;
- reproducing advertising boilerplate;
- training on account content that may have been staff-written;
- weaker controllability when the user's requested content differs from the author's historical topics.

The literature explicitly warns that a single social-media account is not necessarily equivalent to one human author: research comparing owner- and staff-attributed tweets found measurable differences in linguistic topics and style. ¹⁷

How I would review your current workbook/proposal architectures

Once the actual workbook/proposal is available, every architecture should be scored against this rubric:

Question	What we need to establish
What hypothesis does this architecture test?	No architecture should exist merely because a technology is fashionable.
What is its simplest baseline?	Complexity must be justified experimentally.
What input data does it consume?	Research papers, author posts, ontology facts, and evaluation data must not blur together.
Where does author identity enter?	Prompt, retrieval, adapter, embedding, or graph?

Question	What we need to establish
Where does factual knowledge enter?	It should not be conflated with style examples.
What is trained?	Model weights, retriever, classifier, ontology mapper, nothing?
What data are seen during training?	Prevent contamination and evaluation leakage.
What metric demonstrates success?	Style quality and factual correctness require different metrics.
What ablation proves each component helps?	Remove graph, remove style retrieval, raw vs cleaned, etc.
What is computationally feasible?	Capstone architecture should be executable repeatedly, not merely theoretically attractive.
Can the experiment be reproduced?	Config, dataset version, seed, model, prompt and commit must be recorded.

That last point is especially appropriate for an SJSU culminating project because current ADS assessment materials explicitly emphasize executing the complete project lifecycle and producing quality project artifacts. ⁶

How the ontology-grounded RAG branch should actually work

A key distinction needs to be preserved in your proposal:

GraphRAG and ontology-grounded RAG are related but not synonymous.

Microsoft GraphRAG creates a knowledge graph from text, groups it into communities, generates community summaries, and supports local and global query strategies. ¹³ OG-RAG, by contrast, explicitly grounds document knowledge using a **domain ontology** and constructs ontology-grounded hypergraph structures for retrieval. ¹¹

Therefore, simply installing Microsoft GraphRAG and saying “ontology RAG” would be conceptually weak if your professor specifically asked you to explore ontological RAG.

A reasonable project ontology

For the project you described, I would start small.

Author
Post
Topic
Entity

```
Claim
Stance
Event
Organization
Product
Promotion
Platform
Source
StyleProfile
```

Potential relationships:

```
Author    --AUTHORED-->    Post
Post      --ABOUT_TOPIC--> Topic
Post      --MENTIONS-->    Entity
Post      --ASSERTS-->     Claim
Author    --EXPRESSES_STANCE_ON--> Topic
Claim     --ABOUT-->      Entity
Post      --REFERS_TO-->   Event
Post      --PROMOTES-->    Product
Promotion --APPEARS_IN-->   Post
Post      --POSTED_ON-->   Platform
Claim     --SUPPORTED_BY--> Source
Author    --HAS_STYLE_PROFILE--> StyleProfile
Post      --REPLY_TO-->    Post
Post      --QUOTES-->      Post
```

Do **not** begin with 80 classes and 200 predicates. OG-RAG itself depends on the availability of domain ontological knowledge, and the paper lists automated ontology abstraction as an area for future work; ontology creation is therefore not a free or trivial step. ¹⁹

For a capstone, a manually reviewed minimum viable ontology will give you a cleaner experimental variable.

Query-time retrieval

A practical ontology-aware retrieval sequence would be:

```
query
↓
identify target author + entities + topic
↓
dense/lexical retrieval obtains candidate source posts/chunks
↓
map candidates/query entities onto ontology
↓
```

```
expand selected graph relations
↓
retrieve connected claims/posts/events
↓
rerank evidence
↓
assemble factual/persona context with provenance
```

You can then compare that directly against:

```
query → dense retrieval → top-k chunks
```

That makes the **ontology the independent architectural variable** rather than changing ten things at the same time.

KG²RAG is useful support for this design because its core idea is to use fact-level graph relationships between chunks to improve the diversity and coherence of retrieved context. ¹²

Keep the style branch out of the factual graph traversal

The generation prompt should make the distinction explicit:

```
FACTUAL/PERSONA EVIDENCE:
[retrieved ontology/vector material]

STYLE EXEMPLARS:
[author posts selected for linguistic similarity]

INSTRUCTIONS:
Use factual/persona evidence to determine what may be asserted.
Use style exemplars only to guide expression, tone, syntax, vocabulary,
punctuation, and discourse patterns.

Do not infer a factual claim merely because it appears in a style example.
Do not reproduce long phrases from an exemplar.
```

That gives you much cleaner interpretability.

When the answer is wrong, you can ask:

- Was the fact missing from retrieval?
- Did ontology traversal retrieve the wrong relationship?
- Did the model ignore correct evidence?
- Did a stylistic example accidentally inject a claim?

Without that separation, all four failure modes look like “the LLM hallucinated.”

Where Microsoft GraphRAG belongs

I would treat GraphRAG as either:

- a literature/reference architecture; or
- an optional additional comparison if time allows.

Its local search combines structured graph information with raw text, while global search uses hierarchical community reports to answer corpus-level questions. ²⁰

That is attractive when your questions include things like:

“What themes characterize this person's views over their entire history?”

But your **ontology experiment** should retain a manually specified schema so you can credibly say it is ontology-guided rather than merely graph-derived.

Do not skip the simple baseline

This is important enough to emphasize. EMNLP 2025 research found that a simple structure-preserving retrieval baseline matched or outperformed substantially more complicated multi-stage RAG approaches on several long-context QA benchmarks and recommended controlled comparisons under matched token budgets. ³

Your experimental story therefore becomes much stronger if you report:

“Ontology RAG improved persona-consistency by X, multi-hop accuracy by Y, and retrieval recall by Z while costing Q additional tokens/latency.”

rather than:

“We implemented ontology RAG because it is more sophisticated.”

Cleaning social-media data without destroying the author's style

This is probably the most important data-engineering issue you raised.

Do not make “cleaning” synonymous with deleting everything that looks unusual.

For author modeling, punctuation, capitalization, emoji, hashtags, contractions, spelling habits, sentence boundaries, word choice, syntax, and platform-specific behavior can all contain stylistic signal. Authorship research has repeatedly shown that lexical, syntactic, and other linguistic features contribute to style representation, while social-media preprocessing studies often normalize URLs/usernames precisely because those artifacts can interfere with downstream text modeling. ²¹

Preserve an immutable raw layer

Use:

```
data/raw/  
data/interim/  
data/processed/  
data/annotations/
```

Raw records should never be overwritten.

Each processed record should retain something like:

```
{  
  "post_id_hash": "...",  
  "author_id": "...",  
  "platform": "...",  
  "timestamp": "...",  
  "raw_text_ref": "...",  
  "normalized_text": "...",  
  "content_type": "organic",  
  "promo_score": 0.04,  
  "promo_flags": [],  
  "authorship_confidence": 0.93,  
  "duplicate_cluster": null,  
  "split": "train",  
  "pipeline_version": "clean-v3"  
}
```

That gives you reversibility.

Label promotions rather than permanently throwing them away

Your promotional contamination problem should become an explicit modeling task:

```
organic  
promotion_or_ad  
repost_or_quote  
reply  
announcement  
automated_or_template  
uncertain
```

A first-pass promotional score can use interpretable signals such as:

```
affiliate / tracking URLs
"link in bio"
"shop now"
"order now"
"available now"
"limited time"
"discount"
"use code"
"promo code"
"tickets"
"subscribe"
"sponsored"
"#ad"
percentage discounts
repeated calls to action
high similarity to repeated campaign templates
```

But those should initially be **flags**, not unquestionable truth.

Promotion detection is difficult because campaigns can be coordinated or template-like while organic posts can contain many of the same tokens. Prior social-network spam research has exploited repeated content and URL behavior as useful campaign signals, illustrating why duplicate/template structure can be informative in addition to keywords. ²²

Deduplicate before model training

I would detect:

1. exact duplicates;
2. normalized duplicates;
3. near-duplicate promotional templates;
4. cross-author duplicate campaigns;
5. quote/repost copies.

A repeated sponsor caption appearing twenty times should not count as twenty independent demonstrations of an author's style.

Likewise, duplicates must be grouped before train/validation/test splitting; otherwise a near-identical post can leak between training and evaluation and give you an artificially optimistic model.

The critical complication: promotional posts may not have been written by the account owner

This is not hypothetical. Research specifically comparing owner- and staff-attributed tweets found that social-media research can be wrong to assume all posts from one account have the same underlying author and found linguistic differences between the categories. ¹⁷

That means your cleaning objective is actually closer to:

Find text that is likely to represent the target individual's personal writing distribution.

rather than:

Remove advertisements.

Promotional posts are merely one high-risk category.

I would therefore add an `authorship_confidence` field based on:

- promotion status;
- template similarity;
- posting pattern;
- linguistic distance from an author's normal corpus;
- source metadata;
- manual review where possible.

Do not remove topic leakage blindly either

Authorship research finds that topic and style interact and that proper nouns/topic words can heavily influence attribution. ¹⁸

That means you need two representations:

Generation text

Preserve much of the author's genuine text because it is needed for style imitation.

Style-analysis representation

Optionally mask high-content fields such as:

```
<URL>
<USER>
<EMAIL>
<PRODUCT>
<LOCATION>
```

and evaluate whether the remaining model still distinguishes authors.

That gives you a way to determine whether your “style model” learned stylistic identity or simply learned that one person always discusses a particular basketball team, cryptocurrency, political figure, or brand.

Build multiple cleaned corpora instead of betting everything on one definition

I recommend at least:

CORPUS_RAW

Everything after basic parsing.

CORPUS_ORGANIC_STRICT

High-confidence personally authored, non-promotional content.

CORPUS_ORGANIC_BROAD

Organic + uncertain but plausible author-written material.

CORPUS_PROMOTIONAL

Ads, sponsorships, repetitive campaigns, affiliate content.

Then test the hypothesis:

H-cleaning: Models conditioned/trained on `ORGANIC_STRICT` should better reproduce author-specific writing style than models using the unfiltered corpus.

That transforms cleaning from an undocumented preprocessing choice into a **research contribution and ablation**.

Do not evaluate style with BLEU/ROUGE alone

Recent work specifically examining personalized text-generation evaluation questions the adequacy of commonly used overlap metrics such as BLEU and ROUGE and finds advantages to combining several evaluation approaches rather than depending on a single evaluator. ²³ The EMNLP 2025 author-imitation study similarly uses an ensemble including authorship attribution, authorship verification, style matching, and AI detection. ¹⁴

I would evaluate style using a combination of:

Dimension	Example measurement
Author discriminability	Can an authorship classifier identify the intended author?
Verification	Does an AV model judge generated/original pairs as same-author-like?
Stylometry	Sentence length, punctuation, POS patterns, lexical diversity, emoji, capitalization, etc.
Style representation	Embedding distance to target author's held-out posts

Dimension	Example measurement
Blind human judgment	Which output better resembles held-out author writing?
Exact-copy rate	Detect memorization / copied n-grams
Content preservation	Did style transformation preserve requested meaning?

The 2026 ACL work on style representations is also particularly interesting for your project: it found that eliciting an interpretable style representation and using it for steering outperformed baselines that directly supplied target text in its evaluated settings. ¹⁵ That suggests an architecture worth testing between simple exemplar prompting and fine-tuning:

```

clean posts
  ↓
extract explicit style profile
  ↓
style profile + retrieved examples
  ↓
generation

```

A per-author style profile might contain:

```

Typical sentence length
Fragment frequency
Contraction preference
Capitalization patterns
Emoji frequency and position
Hashtag behavior
Preferred discourse markers
Question/exclamation frequency
Common syntactic structures
Degree of directness
Humor / rhetorical tendencies
Typical paragraph structure

```

The profile should be learned from **training posts only** to avoid test leakage.

Social-media research governance needs to be in the project

Because your data are scraped from people's social-media profiles, I would add an explicit `DATA_GOVERNANCE.md` or section in `DATA_PROVENANCE.md`.

Do **not** assume that “it was publicly visible online” automatically resolves research-ethics questions. Federal guidance concerning internet research explicitly discusses harvesting, mining, profiling, scraping, blogs,

and social-media posts and raises questions about when ostensibly online information should be regarded as private or reasonably expected to remain private. ²⁴ More recent HHS discussion of AI research similarly notes that AI frequently uses previously collected social-media material and cautions against simplistic public/private assumptions. ²⁵

SJSU states that its IRB determines whether activities meet the federal definitions of research and human subjects, and it instructs investigators not to begin applicable human-subject data collection before receiving the appropriate formal determination/approval. ²⁶

Since you already have scraped material, your documentation should explicitly record:

```
Source platforms
Acquisition date
Collection method
Whether profiles/posts were public
Platform terms/restrictions considered
Whether identifiers are retained
Whether authors are identifiable in publications
IRB / exemption / non-human-subject determination status
Storage protections
Who has access
Retention/deletion plan
Whether raw data may be committed to Git
```

I would keep the raw scraped corpus **out of a public Git repository** and use hashed/internal identities in derived experimentation wherever the research does not require publication of names. That is a conservative data-governance design consistent with the identifiability and privacy concerns emphasized in SJSU and HHS guidance. ²⁷

Experimental design, evaluation, and the path I would record in your project

The project can now be organized around a compact set of falsifiable research questions rather than an expanding list of technologies.

Proposed research questions

Style question

Does removal/down-weighting of promotional, templated, and potentially third-party-authored social-media content improve target-author style fidelity?

Retrieval question

Does author-specific exemplar retrieval outperform random or fixed few-shot author examples?

Ontology question

Does ontology-grounded retrieval improve persona/factual consistency, multi-hop reasoning, and evidence retrieval relative to ordinary vector retrieval?

Adaptation question

Does PEFT/LoRA improve author-style fidelity over exemplar retrieval sufficiently to justify its added cost and memorization risk?

Combined-system question

Can factual/persona grounding and author-style conditioning be optimized independently without degrading either content correctness or style fidelity?

Those questions are consistent with current evidence that ontology/KG approaches primarily address structured retrieval and reasoning, while personalized author-style generation remains a distinct challenge requiring its own evaluation. ²⁸

Experimental matrix

A strong capstone experiment would look roughly like this:

ID	Knowledge	Style	Data	Purpose
E0	None	Few-shot	Raw	Lowest baseline
E1	None	Few-shot	Organic strict	Cleaning ablation
E2	None	Retrieved exemplars	Organic strict	Style-RAG
E3	Vector RAG	Retrieved exemplars	Organic strict	Standard RAG baseline
E4	Ontology + vector RAG	Retrieved exemplars	Organic strict	Main ontology experiment
E5	Ontology + vector RAG	Explicit style profile + exemplars	Organic strict	Structured-style experiment
E6	Ontology + vector RAG	PEFT/LoRA + exemplars	Organic strict	Advanced architecture

Not every experiment needs to become a production architecture. Several exist precisely to isolate variables.

Keep experimental conditions matched

Across architecture comparisons, hold constant wherever possible:

```
base generation model
generation temperature
maximum output length
question set
test authors
retrieval corpus
evaluation model
token/context budget
prompt requirements
random seed where applicable
```

The strong-baseline RAG literature specifically emphasizes matched token budgets when judging whether sophisticated retrieval systems actually produce meaningful improvement. ³

Retrieval metrics

For retrieval itself:

```
Recall@k
Precision@k
MRR
nDCG@k
multi-hop evidence coverage
source diversity
```

For ontology retrieval, add:

```
entity-linking accuracy
relation/path accuracy
ontology coverage
percentage of generated factual claims traceable to retrieved nodes/chunks
```

Generation metrics

For factual/persona performance:

```
answer correctness
faithfulness / groundedness
citation/evidence correctness
claim support rate
contradiction rate
appropriate abstention
```

Recent RAG evaluation research explicitly separates correctness and faithfulness/groundedness rather than treating them as interchangeable; work on answer faithfulness also emphasizes the need to evaluate whether statements are actually supported by the retrieved context. ²⁹

For style:

```
authorship attribution
authorship verification
style embedding similarity
stylometric distance
human pairwise preference
semantic/content preservation
```

For safety and overfitting:

```
exact copied n-grams
longest common substring
training-post nearest-neighbor similarity
PII leakage
verbatim memorization rate
```

For systems engineering:

```
retrieval latency
generation latency
input tokens
output tokens
graph construction cost
index size
training GPU-hours if applicable
estimated inference cost
```

A proper evaluation dataset

Create a locked evaluation set that includes different query types:

Style-only
Simple author fact
Single-hop persona question
Multi-hop relationship question
Historical/time-sensitive question
Topic/stance question
Insufficient-evidence question
Off-topic question
Adversarial contradiction
Content very different from author's training topics

The last two categories are particularly valuable because they distinguish **real stylistic generalization** from memorizing the author's common subject matter. Authorship research has documented strong topic/style interactions, which makes cross-topic evaluation important. ¹⁸

Record every experiment reproducibly

I would give each run an immutable ID:

EXP-0027

and store:

```
experiment_id
timestamp
git_commit
data_version
cleaning_version
ontology_version
author_split_version
base_model
adapter_version
embedding_model
retriever_type
top_k
reranker
prompt_version
temperature
seed
evaluation_set
metrics
notes
```

Your research paper can then say:

"Architecture A2 corresponds to experiments EXP-0034 through EXP-0041 using ontology v0.4 and clean-v3."

instead of attempting to reconstruct settings from notebook history.

The project plan I would now consider canonical

The path I would record in `docs/project/PLAN.md` is:

PHASE: CONTEXT CONSOLIDATION

- Establish AGENTS.md.
- Establish PROJECT_CONTEXT.md.
- Establish PROJECT_STATE.md.
- Import existing architectural decisions from proposal/workbook.
- Record unresolved contradictions instead of silently choosing one.

PHASE: RESEARCH SYNTHESIS

- Inventory every paper under docs/research.
- Create PAPER_MANIFEST.
- Classify papers by:
 - RAG
 - ontology/KG
 - personalization/style
 - social-media preprocessing
 - evaluation
 - ethics/governance
- Build CLAIMS_MATRIX.
- Update architecture rationale with paper IDs.

PHASE: DATA AUDIT

- Freeze raw scrape.
- Build data dictionary.
- Quantify authors, posts, tokens, platforms, dates.
- Measure missingness.
- Find exact duplicates.
- Find near-duplicate campaigns.
- Measure URL/hashtag/@mention prevalence.
- Discover repeated promotional phrases.
- Create content-type annotation schema.
- Examine likelihood of account-owner vs staff/agency content.

PHASE: CLEANING PIPELINE

- Normalize without destroying stylistic signals.
- Label promotions instead of immediately deleting them.
- Canonicalize platform artifacts where appropriate.
- Create RAW, ORGANIC_STRICT, ORGANIC_BROAD and PROMOTIONAL corpora.
- Create leakage-safe train/validation/test splits.
- Version the pipeline.

PHASE: BASELINES

- Implement fixed/few-shot author exemplar baseline.
- Implement retrieved-exemplar Style-RAG.
- Lock evaluation harness.
- Establish initial style metrics.

PHASE: STANDARD KNOWLEDGE RAG

- Implement ordinary vector retrieval.
- Add factual/persona evaluation.
- Establish RAG baseline under controlled token budget.

PHASE: ONTOLOGY

- Define minimum viable ontology.
- Manually review ontology concepts and relations.
- Map corpus entities/topics/claims into ontology.
- Implement ontology-guided retrieval.
- Preserve provenance to original posts/chunks.
- Evaluate ontology vs vector-only retrieval.

PHASE: ADVANCED PERSONALIZATION

- Extract explicit style profiles.
- Evaluate profile + exemplar architecture.
- If training data and compute justify it, implement PEFT/LoRA.
- Test memorization and cross-topic generalization.

PHASE: ABLATIONS

- raw vs cleaned data
- random vs retrieved exemplars

- vector vs ontology retrieval
- ontology without style retrieval
- style retrieval without ontology
- explicit style profile vs exemplars
- PEFT vs no PEFT

PHASE: FINAL ARCHITECTURE SELECTION

- Select architecture from measured evidence.
- Do not automatically choose the most complex system.
- Document rejected alternatives and results.

PHASE: CAPSTONE ARTIFACTS

- Final architecture diagram.
- Data pipeline diagram.
- Ontology/schema documentation.
- Experimental results.
- Reproducibility instructions.
- Limitations.
- Privacy/ethics documentation.
- Demonstration.

That progression turns the professor's ontology suggestion into a genuine experiment rather than a feature bolted onto the final system.

What I would consider the likely final architecture today

Subject to reviewing your actual proposal, workbook, papers, and repository, my current best design is:

A two-channel personalized generation system in which an ontology-aware hybrid retriever supplies factual/persona evidence while a separate author-style retriever supplies high-confidence organic writing exemplars; the generator is evaluated first without fine-tuning, then optionally with a PEFT/LoRA style adapter.

The reasons are evidence-based:

Ontology-grounded retrieval has demonstrated advantages for structured, fact-oriented retrieval and reasoning in the recent OG-RAG literature. ¹¹ KG-guided retrieval provides additional evidence that relationships among facts/chunks can improve retrieval context organization. ¹² Recent research simultaneously warns that author-style imitation remains imperfect and should be evaluated with several complementary measures. ¹⁰ PEFT is worth testing as a style-control mechanism, but it should be an experimental branch rather than an assumed requirement. ¹⁶ And strong recent RAG baselines demonstrate why the ontology architecture must be judged against a simpler retrieval system under comparable conditions. ³

The architectural principle I would make non-negotiable in your project context is therefore:

RAG decides what evidence the model may use. Style modeling decides how the model expresses it. Data cleaning decides what evidence genuinely represents the target author. Evaluation must measure all three separately.

That is the context I would want both myself and Cursor to inherit before either of us makes significant implementation decisions.

-
- 1 18 21 What represents “style” in authorship attribution? - ACL Anthology
https://aclanthology.org/C18-1238/?utm_source=chatgpt.com
 - 2 9 11 19 28 OG-RAG: Ontology-grounded retrieval-augmented generation for large language models - ACL Anthology
https://aclanthology.org/2025.emnlp-main.1674/?utm_source=chatgpt.com
 - 3 Stronger Baselines for Retrieval-Augmented Generation with Long-Context Language Models - ACL Anthology
https://aclanthology.org/2025.emnlp-main.1656/?utm_source=chatgpt.com
 - 4 7 Projects in ChatGPT | OpenAI Help Center
https://help.openai.com/en/articles/10169521-projects-in-chatgpt?utm_source=chatgpt.com
 - 5 Cursor – Rules
https://docs.cursor.com/context/rules?utm_source=chatgpt.com
 - 6 Course Assessment
https://www.sjsu.edu/applied-data-science/docs/masdi_course_assessment_table.pdf?utm_source=chatgpt.com
 - 8 Cursor – Working with Context
https://docs.cursor.com/en/guides/working-with-context?utm_source=chatgpt.com
 - 10 14 Catch Me If You Can? Not Yet: LLMs Still Struggle to Imitate the Implicit Writing Styles of Everyday Authors - ACL Anthology
https://aclanthology.org/2025.findings-emnlp.532/?utm_source=chatgpt.com
 - 12 Knowledge Graph-Guided Retrieval Augmented Generation - ACL Anthology
https://aclanthology.org/2025.naacl-long.449/?utm_source=chatgpt.com
 - 13 Welcome - GraphRAG
https://microsoft.github.io/graphrag/?utm_source=chatgpt.com
 - 15 Interpreting Style Representations via Style-Eliciting Prompts - ACL Anthology
https://aclanthology.org/2026.findings-acl.2039/?utm_source=chatgpt.com
 - 16 Customizing Large Language Model Generation Style using Parameter-Efficient Finetuning
https://arxiv.org/abs/2409.04574?utm_source=chatgpt.com
 - 17 Analyzing Linguistic Differences between Owner and Staff Attributed Tweets - ACL Anthology
https://aclanthology.org/P19-1274/?utm_source=chatgpt.com
 - 20 Overview - GraphRAG
https://microsoft.github.io/graphrag/query/overview/?utm_source=chatgpt.com

22 An Unsupervised Approach to Detect Spam Campaigns that Use Botnets on Twitter

https://arxiv.org/abs/1804.05232?utm_source=chatgpt.com

23 Evaluating Style-Personalized Text Generation: Challenges and Directions

https://arxiv.org/abs/2508.06374?utm_source=chatgpt.com

24 Attachment B: Considerations and Recommendations concerning Internet Research and Human Subjects Research Regulations, with Revisions | HHS.gov

https://www.hhs.gov/ohrp/sachrp-committee/recommendations/2013-may-20-letter-attachment-b/index.html?utm_source=chatgpt.com

25 IRB Considerations on the Use of Artificial Intelligence in Human Subjects Research | HHS.gov

https://www.hhs.gov/ohrp/sachrp-committee/recommendations/irb-considerations-use-artificial-intelligence-human-subjects-research/index.html?utm_source=chatgpt.com

26 27 Humans Subjects Research - Institutional Review Board (IRB) | Office of Research

https://www.sjsu.edu/research/research-compliance/irb/index.php?utm_source=chatgpt.com

29 Generate but Verify: Answering with Faithfulness in RAG-based Question Answering - ACL Anthology

https://aclanthology.org/2025.ijcnlp-long.56/?utm_source=chatgpt.com